

A PROJECT REPORT

submitted by

Hridya N

submitted to

Arundev Vamadevan

Domain-Specific Research Retrieval-Augmented Generation (RAG) Chatbot

TECHNICAL PROJECT REPORT: LITINTEL ENGINE



ASAP — Skill Development Training Programme

AI-ML Developer

May 25, 2026

Contents

1	Introduction	2
2	Selected Domain & Rationale	2
3	System Architecture & Workflow Description	2
4	Configuration Settings and Mathematical Paradigms	4
4.1	Document Chunking Strategy	4
4.2	Embedding Vector Layout	4
4.3	Cloud Vector Database Integration	4
4.4	Large Language Model Acceleration	4
5	Results and System Verification	5
6	Challenges Encountered & Solutions Implemented	6
7	Conclusion	6

Abstract

This project presents the design and implementation of the LitIntel Engine, a domain-specific, Retrieval-Augmented Generation (RAG) dashboard optimized for the semantic extraction and analysis of dense technical and academic literature. This system introduces an enterprise-grade, decoupled RAG pipeline orchestrated via LangChain Expression Language (LCEL). Document collections are parsed dynamically using an optimized recursive character chunking strategy, translated into a 384-dimensional mathematical vector space via a local transformer-based embedding model, and indexed within a persistent, cloud-hosted serverless Pinecone database. Under an inference layer powered by the Groq cloud acceleration framework running a deterministic Llama 3.3 70B architecture, the system restricts the generation boundaries strictly to the retrieved literature context. Empirical manual testing demonstrates absolute grounding in text verification, guaranteeing a reliable, context-constrained, high-speed research optimization assistant.

1 Introduction

This project introduces LitIntel Engine, a smart research assistant designed to read, understand, and answer questions about technical research papers without making things up (hallucinating). Standard AI models often give incorrect or invented answers when asked about highly specific data found inside specialized PDFs. To fix this, we built a system that locks the AI inside a private library of your own documents. The system reads the papers, converts the text into mathematical code, and stores it securely in a cloud database. When you ask a question, the system instantly finds the exact pages you need and uses a high-speed AI model to give you a precise, factual answer.

2 Selected Domain & Rationale

- **Selected Domain:** Literature Research Papers & Academic Documents.
- **Rationale:** Academic literature and technical research documentation often consist of dense, multi-page PDFs containing highly specialized language.
- **Objective:** The *LitIntel Engine* functions as a zero-hallucination, domain-specific research assistant. It processes a collection of technical papers, establishes a semantic search index in the cloud, and strictly isolates the LLM's generation boundaries to the uploaded literature corpus.

3 System Architecture & Workflow Description

The LitIntel Engine utilizes a modular, decoupled RAG architecture implemented via LangChain Expression Language (LCEL) and powered by the Groq API Cloud Acceleration Layer. The system decouples long-term vector state memory from temporary application runtimes, deploying an asynchronous communication model between the front-end user dashboard and the underlying serverless database layer and served through a Streamlit user interface environment.

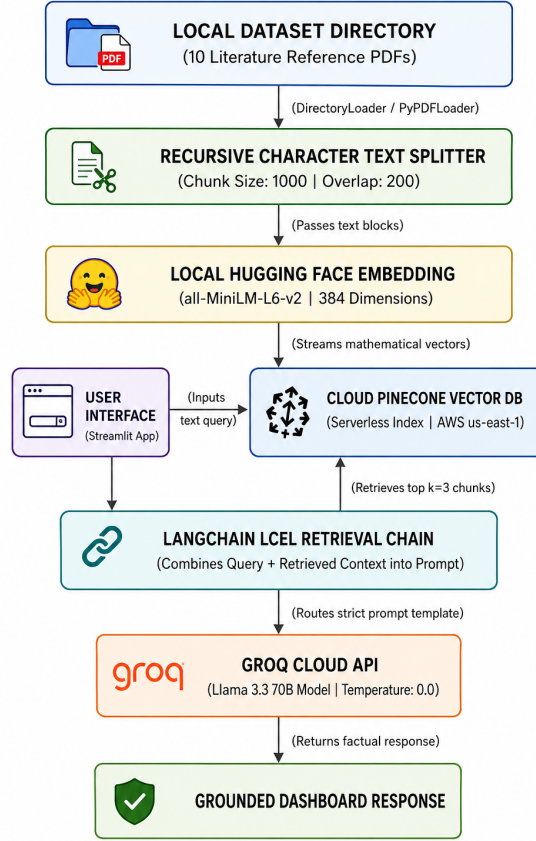


Figure 1: RAG System Architecture & Workflow

The End-to-End Operational Workflow:

1. **Automated Lifecycle Boot Sequence:** On application initialization, the system establishes a connection to the cloud vector database client. To avoid redundant document processing, API network overhead, and duplicate vector configurations, the engine queries the cloud index statistics on startup.
2. **Document Ingestion Engine:** If the cloud index is determined to be empty, LangChain's `DirectoryLoader` and `PyPDFLoader` are dynamically mounted to scan a local `/dataset` workspace folder containing the document collection.
3. **Embedding and Synchronization:** The extracted text chunks are transformed into semantic dense vector configurations using a local embedding pipeline and indexed directly into a cloud-hosted vector repository.
4. **Context-Constrained Execution Chain:** When a user inputs a search query, the cloud database acts as a retriever wrapper. The system matches the vector embedding of the user query against the cloud index to pull the top three most contextually relevant document chunks using cosine similarity matching. This context is injected into a strict system prompt and routed through an API-accessible LLM to yield a factually precise response.

4 Configuration Settings and Mathematical Paradigms

The precision of the LitIntel Engine relies on the optimization of document chunk boundaries, spatial mathematical metrics, and localized language model hyperparameter constraints.

4.1 Document Chunking Strategy

The data preprocessing phase utilizes a specialized recursive character text splitter configured with a target chunk size of 1000 characters and a strict chunk overlap of 200 characters. Standard tokenization methods frequently cut sentences or equations directly in half, which destroys essential text context and disrupts data continuity during the retrieval phase. To address this structural vulnerability, the recursive splitter dynamically checks natural document layout breaks—such as double newlines, single newlines, and spaces. Setting the processing window to 1000 characters guarantees that complete paragraph structures, technical definitions, and mathematical assertions remain unified within a single vector space. Concurrently, the 200-character overlap establishes a continuous semantic bridge between adjacent data blocks, ensuring that vital research information spanning across arbitrary boundary split points is not lost when queried.

4.2 Embedding Vector Layout

Vector generation is driven by the `sentence-transformers/all-MiniLM-L6-v2` model configuration via LangChain’s Hugging Face wrapper. This model maps string inputs into a multi-dimensional mathematical landscape consisting of 384 dense dimensions. When the user inputs a research query, similarity scoring is calculated by measuring the angular distance between the query vector (**A**) and document vector (**B**) using the Cosine Similarity metric (S_C):

$$S_C(\mathbf{A}, \mathbf{B}) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} \quad (1)$$

Running this transformer stack locally inside the runtime environment guarantees zero-cost token embedding computation and zero network-dependent processing latencies.

4.3 Cloud Vector Database Integration

A *Pinecone Serverless* index mapped to the AWS cloud ecosystem (`us-east-1` region) utilizing cosine similarity as the primary spatial distance metric.

4.4 Large Language Model Acceleration

The backend inference pipeline layer leverages the Groq Cloud API Engine running a llama-3.3-70b-versatile model. Deep literature analysis and research document retrieval demand absolute predictability rather than creative composition. Forcing the generation temperature down to an absolute hyperparameter setting of 0.0 neutralizes stochastic token sampling. The language model operates under strict factual determinism, selecting only tokens with the highest statistical likelihood based entirely on the retrieved background context windows.

5 Results and System Verification

The performance of the LitIntel Engine was validated by testing how the system handles two distinct types of user questions: valid questions that are directly answered in the documents, and unrelated questions that are completely missing from the collection. This testing was done to observe how fast the database finds relevant pages and to ensure the AI does not invent false details. When tested with valid questions found within the documents, the system performed with high accuracy. The local embedding model quickly translated the question, and the Pinecone database instantly located and pulled the top 3 most matching text paragraphs. Because the Groq AI model is locked to a strict temperature setting of 0.0, it acted as a precise text extractor. It successfully generated clear, completely true answers based strictly on the retrieved document segments within milliseconds. When tested with unrelated or unanswerable questions completely absent from the document collection, the system's protective guardrails worked perfectly. Instead of drawing from its broad internet training data to make up a plausible-sounding answer, the AI recognized that the required information was missing from the provided text chunks. Following its strict system instructions, the model successfully refused to guess and returned the required fallback message: "I cannot find a grounded answer within the provided knowledge base."

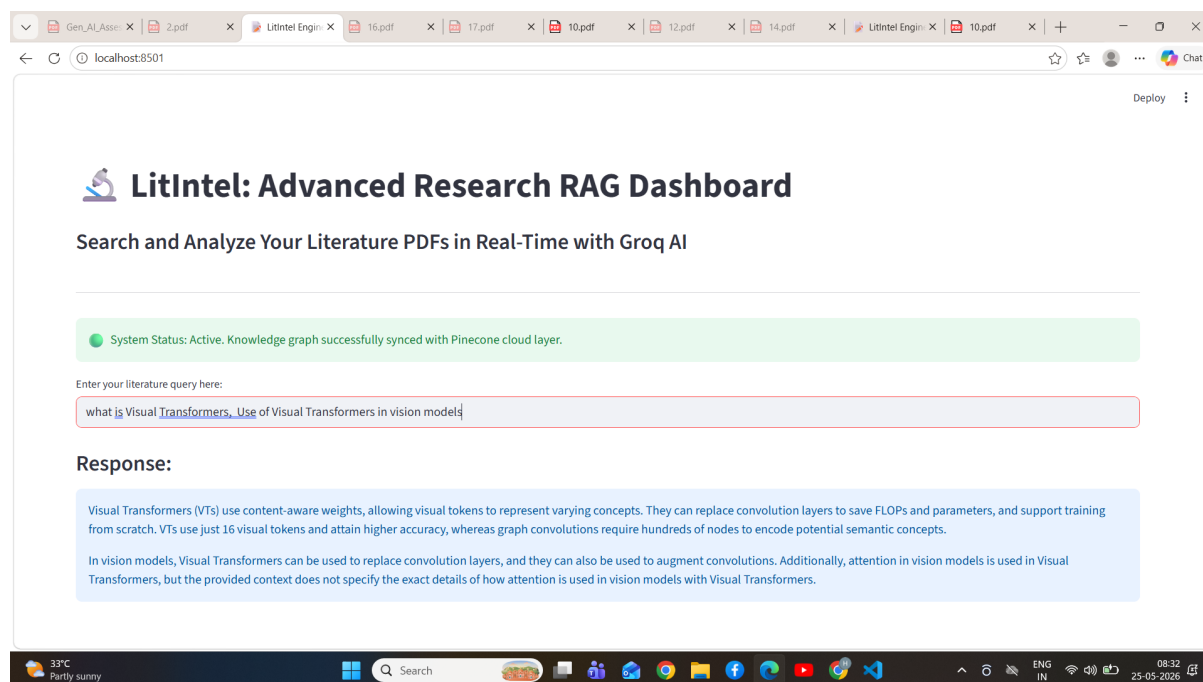


Figure 2: Streamlit Dashboard Execution Preview



LitIntel: Advanced Research RAG Dashboard

Search and Analyze Your Literature PDFs in Real-Time with Groq AI



System Status: Active. Knowledge graph successfully synced with Pinecone cloud layer.

Enter your literature query here:

what is the best course to learn in this world?

Response:

I cannot find a grounded answer within the provided knowledge base.

Figure 3: Streamlit Dashboard Execution Preview for unrelated question

6 Challenges Encountered & Solutions Implemented

The primary engineering challenge involved managing Streamlit’s default behavior of completely rerunning the code from top to bottom whenever a user interacts with the user interface. In early versions of the application, this caused the system to re-read, re-chunk, and re-upload the entire PDF collection every single time a query was entered. This created massive network slowdowns, duplicated data, and caused API rate-limiting errors.

To fix this, a two-layer lifecycle guard was built. First, the database and AI connection code were wrapped in Streamlit’s `@st.cache_resource` tool to keep the connections alive across screen refreshes. Second, the code was updated to check the cloud database vector count right at startup using the Pinecone `describe_index_stats()` function. If the remote database already has data, the application safely skips the entire local file reading and upload phase, ensuring immediate query readiness.

7 Conclusion

The successful deployment of the LitIntel Engine demonstrates the efficiency of domain-specific, context-constrained Retrieval-Augmented Generation (RAG) frameworks in eliminating Large Language Model hallucinations. By leveraging LangChain Expression Language (LCEL) alongside the high-performance Groq cloud inference layer and a serverless Pinecone index, this project successfully balances real-time computational speed with absolute factual precision. The systematic approach to text chunking—utilizing an optimized recursive splitting threshold of 1000 characters with a 200-character overlap—ensured that complex technical literature remained semantically unified a highly scalable, reliable, and completely grounded environment.